

9.2.7

RAVULA SHASHANK REDDY - EE25BTECH11047

October 4, 2025

Question

Find the area of the region bounded by the parabola $y^2 = x$ and the line $2y = x$.

$$\mathbf{x}^T \mathbf{V} \mathbf{x} + 2\mathbf{u}^T \mathbf{x} + f = 0 \quad (1)$$

$$\mathbf{V} = \begin{pmatrix} 0 & 0 \\ 0 & 1 \end{pmatrix}, \quad \mathbf{u} = \begin{pmatrix} -\frac{1}{2} \\ 0 \end{pmatrix}, \quad f = 0 \quad (2)$$

$$(3)$$

$$\mathbf{x} = \mathbf{h} + \kappa \mathbf{m}, \quad \kappa \in \mathbb{R} \quad (4)$$

$$\mathbf{h} = \begin{pmatrix} 0 \\ 0 \end{pmatrix}, \quad \mathbf{m} = \begin{pmatrix} 2 \\ 1 \end{pmatrix} \quad (5)$$

$$\mathbf{x}_i = \mathbf{h} + \kappa_i \mathbf{m} \quad (6)$$

Substituting eq-(6) in eq-(1) and Solving the Quadratic

$$\kappa_i = \frac{1}{\mathbf{m}^T \mathbf{V} \mathbf{m}} \left(-\mathbf{m}^T (\mathbf{V} \mathbf{h} + \mathbf{u}) \pm \sqrt{(\mathbf{m}^T (\mathbf{V} \mathbf{h} + \mathbf{u}))^2 - g(\mathbf{h})(\mathbf{m}^T \mathbf{V} \mathbf{m})} \right) \quad (7)$$

Solution

$$g(\mathbf{h}) = \mathbf{h}^T \mathbf{V} \mathbf{h} + 2\mathbf{u}^T \mathbf{h} + f \quad (8)$$

$$\mathbf{V} \mathbf{m} = \begin{pmatrix} 0 \\ 1 \end{pmatrix}, \quad \mathbf{m}^T \mathbf{V} \mathbf{m} = 1 \quad (9)$$

$$\mathbf{V} \mathbf{h} + \mathbf{u} = \begin{pmatrix} -\frac{1}{2} \\ 0 \end{pmatrix}, \quad \mathbf{m}^T (\mathbf{V} \mathbf{h} + \mathbf{u}) = -1 \quad (10)$$

$$g(\mathbf{h}) = 0 \quad (11)$$

$$\kappa_i = -(-1) \pm \sqrt{(-1)^2 - 0} = 1 \pm 1 \quad (12)$$

$$\implies \kappa_1 = 0, \quad \kappa_2 = 2 \quad (13)$$

$$\mathbf{x}_1 = \mathbf{h} + 0\mathbf{m} = \begin{pmatrix} 0 \\ 0 \end{pmatrix}, \quad \mathbf{x}_2 = \mathbf{h} + 2\mathbf{m} = \begin{pmatrix} 4 \\ 2 \end{pmatrix} \quad (14)$$

$$\text{Area} = \int_0^2 (2y - y^2) dy = \left(y^2 - \frac{y^3}{3} \right)_0^2 = 4 - \frac{8}{3} = \frac{4}{3} \quad (15)$$

C Code

```
#include <stdio.h>
#include <math.h>

// Function g(h)
double g(double h[2], double V[2][2], double u[2], double f) {
    double temp1 = h[0]*(V[0][0]*h[0] + V[0][1]*h[1]) +
        h[1]*(V[1][0]*h[0] + V[1][1]*h[1]);
    double temp2 = 2*(u[0]*h[0] + u[1]*h[1]);
    return temp1 + temp2 + f;
}

int main() {
    // Conic parameters for  $y^2 = x$ 
    double V[2][2] = {{0,0},{0,1}};
    double u[2] = {-0.5,0};
    double f = 0;
```

```
// Line parameters  $x = h + k*m$  ( $x = 2y$ )
double h[2] = {0,0};
double m[2] = {2,1};

// Compute intersections using formula
double mtVm = m[0]*(V[0][0]*m[0] + V[0][1]*m[1]) +
              m[1]*(V[1][0]*m[0] + V[1][1]*m[1]);

double Vh_u[2] = {V[0][0]*h[0] + V[0][1]*h[1] + u[0],
                  V[1][0]*h[0] + V[1][1]*h[1] + u[1]};
double mVu = m[0]*Vh_u[0] + m[1]*Vh_u[1];
double disc = mVu*mVu - g(h,V,u,f)*mtVm;

double kappa1 = (-mVu + sqrt(disc))/mtVm;
double kappa2 = (-mVu - sqrt(disc))/mtVm;

double x1[2] = {h[0] + kappa1*m[0], h[1] + kappa1*m[1]};
```

```
double x2[2] = {h[0] + kappa2*m[0], h[1] + kappa2*m[1]};

printf(Intersection points:\n);
printf(x1 = (%.2f, %.2f)\n, x1[0], x1[1]);
printf(x2 = (%.2f, %.2f)\n, x2[0], x2[1]);

// Area bounded (integral from y=0 to y=2 of (2y - y^2) dy)
double area = (pow(2,2) - pow(2,3)/3.0) - (0 - 0);
printf(Bounded Area = %.4f\n, area);

// Generate data for plotting
FILE *fp = fopen(parabola_line.dat,w);
if(fp == NULL) {
    printf(Error opening file.\n);
    return 1;
}
```

```
// Parabola points
for(double y=-3; y<=3; y+=0.05) {
    double x = y*y;
    fprintf(fp,"%lf %lf P\n",x,y); // P for parabola
}
// Line points
for(double y=-3; y<=3; y+=0.05) {
    double x = 2*y;
    fprintf(fp,"%lf %lf L\n",x,y); // L for line
}
fclose(fp);
printf(Data written to parabola_line.dat (columns: x y label)
      .\n);
printf(Use gnuplot to plot:\n);
printf( plot 'parabola_line.dat' using 1:2:(stringcolumn(3))
      with labels\n);
return 0;
}
```


Python Direct

```
import numpy as np
import matplotlib.pyplot as plt
from numpy import linalg as LA

# Local imports
from libs.line.funcs import *
from libs.triangle.funcs import *
from libs.conics.funcs import *

# Setting up plot
fig, ax = plt.subplots()
ax.set_aspect('equal')
ax.grid()

# Conic parameters for  $y^2 = x$ 
V = np.array([0,0],[0,1])
u = np.array([-0.5,0])
f = 0
```

Python Direct

```
# Line parameters  $x = h + k*m$  ( $x = 2y$ )
h = np.array([0,0])
m = np.array([2,1])

# Function g(h)
def g(h,V,u,f):
    return h@V@h + 2*u@h + f

# Compute intersections using formula
mtVm = m@V@m
mVu = m@(V@h + u)
disc = mVu**2 - g(h,V,u,f)*mtVm
kappa1 = (-mVu + np.sqrt(disc))/mtVm
kappa2 = (-mVu - np.sqrt(disc))/mtVm

x1 = h + kappa1*m
x2 = h + kappa2*m
```

Python Direct

```
# Generate full parabola
y_parab = np.linspace(-3,3,400)
x_parab = y_parab**2
plt.plot(x_parab,y_parab,label='$y^2=x$',color='r')

# Generate full line
y_line = np.linspace(-3,3,400)
x_line = 2*y_line
plt.plot(x_line,y_line,label='$x=2y$',color='b')

# Shade the bounded area between y=0 and y=2
y_fill = np.linspace(0,2,200)
x_left = y_fill**2
x_right = 2*y_fill
plt.fill_betweenx(y_fill, x_left, x_right, color='blue', alpha
    =0.3)

# Intersection points
```

Python Direct

```
points = np.vstack((x1,x2)).T
plt.scatter(points[0,:],points[1:],color='k')

# Labels with rounding
labels = [f$({round(points[0,i],1)},{round(points[1,i],1)})$ for
i in range(2)]
for i, txt in enumerate(labels):
    plt.annotate(txt,
                 (points[0,i], points[1,i]),
                 textcoords=offset points,
                 xytext=(0,5),
                 ha='center')

plt.xlabel('$x$')
plt.ylabel('$y$')
plt.legend(loc='best')
plt.title(Parabola  $y^2=x$  and Line  $x=2y$  with Bounded Area)
plt.show()
```

```
import ctypes

# Load shared library
lib = ctypes.CDLL('./libparab_area.so')

# Define function signature
lib.parab_area.argtypes = [ctypes.POINTER(ctypes.c_double),
                           ctypes.POINTER(ctypes.c_double),
                           ctypes.POINTER(ctypes.c_double),
                           ctypes.POINTER(ctypes.c_double),
                           ctypes.POINTER(ctypes.c_double)]

# Prepare variables
x1 = ctypes.c_double()
y1 = ctypes.c_double()
x2 = ctypes.c_double()
y2 = ctypes.c_double()
area = ctypes.c_double()
```

```
import ctypes

# Call function
lib.parab_area(ctypes.byref(x1), ctypes.byref(y1),
               ctypes.byref(x2), ctypes.byref(y2), ctypes.byref(
                   area))

# Print results
print(fIntersection 1: ({round(x1.value,2)}, {round(y1.value,2)})
      )
print(fIntersection 2: ({round(x2.value,2)}, {round(y2.value,2)})
      )
print(fArea = {round(area.value,4)})
```

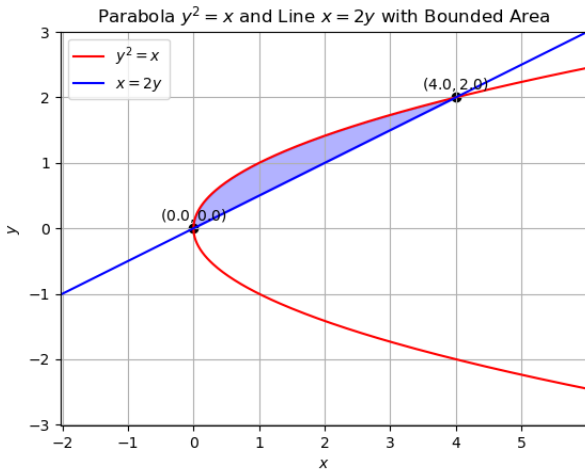


Figure: